



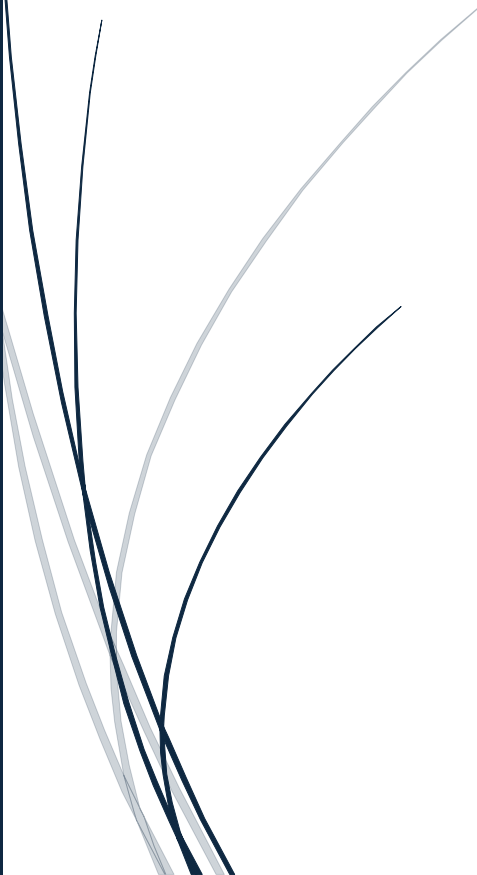
24 de septiembre de 2026

Lab: File path traversal, traversal sequences stripped non-recursively

CNO IV – Seguridad Informática

Profesor: Servando López Contreras
Universidad Politécnica de San Luis Potosí

Luis Felipe Jaramillo Barajas
MATRICULA: 183559



Contenido

1. Introducción	2
2. Objetivos	2
Objetivo general	2
Objetivos específicos	2
3. Preparación del laboratorio	3
4. Fase 1: Interceptación y confirmación del parámetro	3
5. Fase 2: Envío a Repeater y construcción del payload	4
6. Fase 3: Explotación y confirmación	7
6.1 Explicación del payload	8
6.2 Resolución del laboratorio	8
7. Mitigación recomendada	9
8. Conclusiones	9

1. Introducción

Este informe documenta el laboratorio “File path traversal, traversal sequences stripped non-recursively” de PortSwigger. En este caso, la aplicación intenta neutralizar el ataque eliminando del valor recibido las secuencias de retroceso de directorio (“../”), pero lo hace de forma no recursiva: realiza una única pasada de eliminación sobre la cadena, sin repetir el proceso hasta que no quede ninguna secuencia sospechosa.

Esta limitación permite construir un payload que, tras una sola eliminación de “../”, deja expuesta una nueva secuencia “../” válida, lográndose así extraer el contenido del archivo `/etc/passwd` del servidor.

2. Objetivos

Objetivo general

Demostrar la capacidad de evadir un filtro de eliminación de secuencias de path traversal aplicado de forma no recursiva, construyendo un payload que reconstruya la secuencia maliciosa tras el filtrado.

Objetivos específicos

- Identificar el parámetro `filename` como punto de entrada vulnerable.
- Comprender el funcionamiento de un filtro de eliminación no recursiva de secuencias “../”.
- Construir un payload que reconstruya la secuencia de traversal después del filtrado.
- Acceder al contenido del archivo `/etc/passwd` evadiendo el filtro.
- Proponer medidas de mitigación adecuadas.

3. Preparación del laboratorio

El laboratorio contiene una vulnerabilidad de path traversal en la visualización de imágenes de producto de la tienda “We Like To Shop”. La aplicación elimina del nombre de archivo recibido las secuencias de traversal antes de utilizarlo, pero esta eliminación se realiza en una sola pasada (no recursiva). El objetivo es recuperar el contenido de `/etc/passwd`.

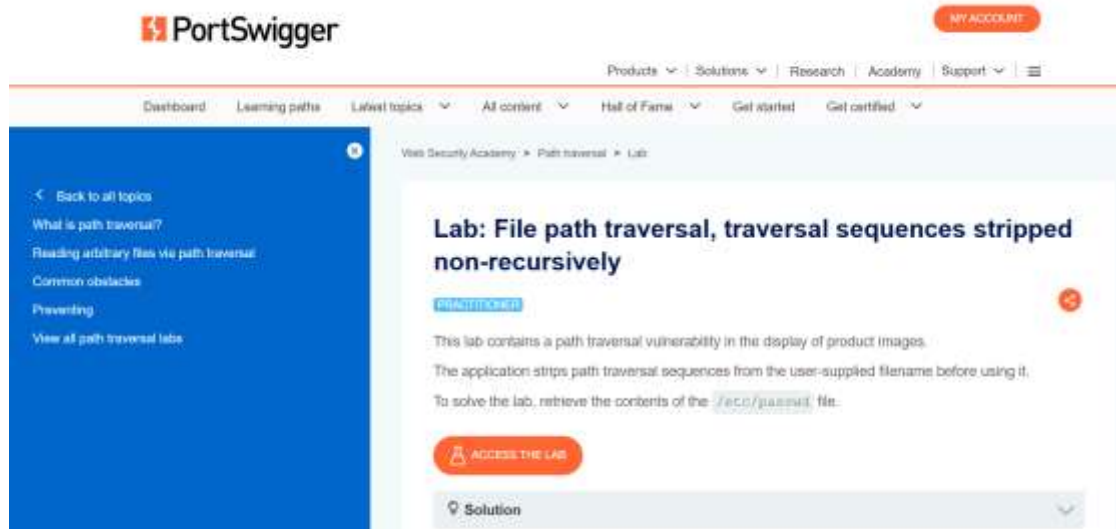


Figura 1. Descripción del laboratorio “Traversal sequences stripped non-recursively” en PortSwigger.

Concepto teórico: filtrado no recursivo. Un filtro no recursivo localiza y elimina las apariciones de un patrón (por ejemplo “../”) en una única pasada sobre la cadena de entrada. El problema aparece cuando, tras eliminar una ocurrencia, los caracteres que quedan a ambos lados se combinan y forman una nueva ocurrencia del mismo patrón que el filtro ya no vuelve a examinar. Un filtro correcto debería repetir la eliminación hasta alcanzar un punto fijo, es decir, hasta que una nueva pasada no produzca cambios.

4. Fase 1: Interceptación y confirmación del parámetro

Con el proxy de Burp Suite activo, se navegó por el catálogo de “We Like To Shop”. En el WebSocket de la aplicación se observó el mensaje inicial de sincronización del laboratorio, y en el historial HTTP se confirmó el mismo patrón de petición para cargar imágenes de producto.

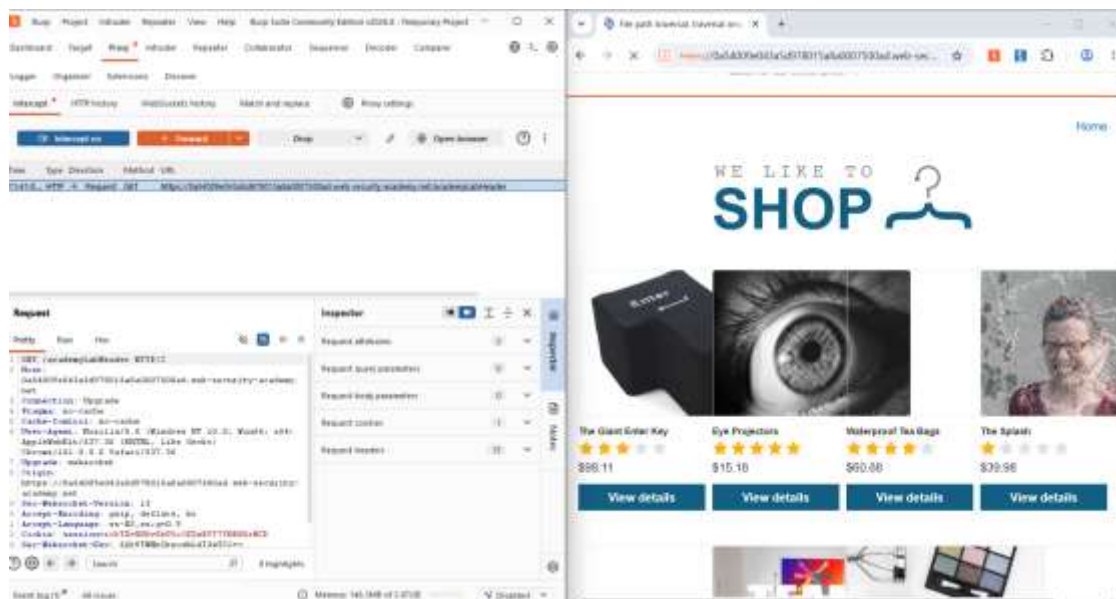


Figura 2. Tráfico inicial capturado en Burp, incluyendo el mensaje WebSocket “PING” de la página del laboratorio.

Al revisar el historial de peticiones HTTP (HTTP history) se localizaron las solicitudes GET /image?filename=N.jpg correspondientes a cada producto del catálogo.

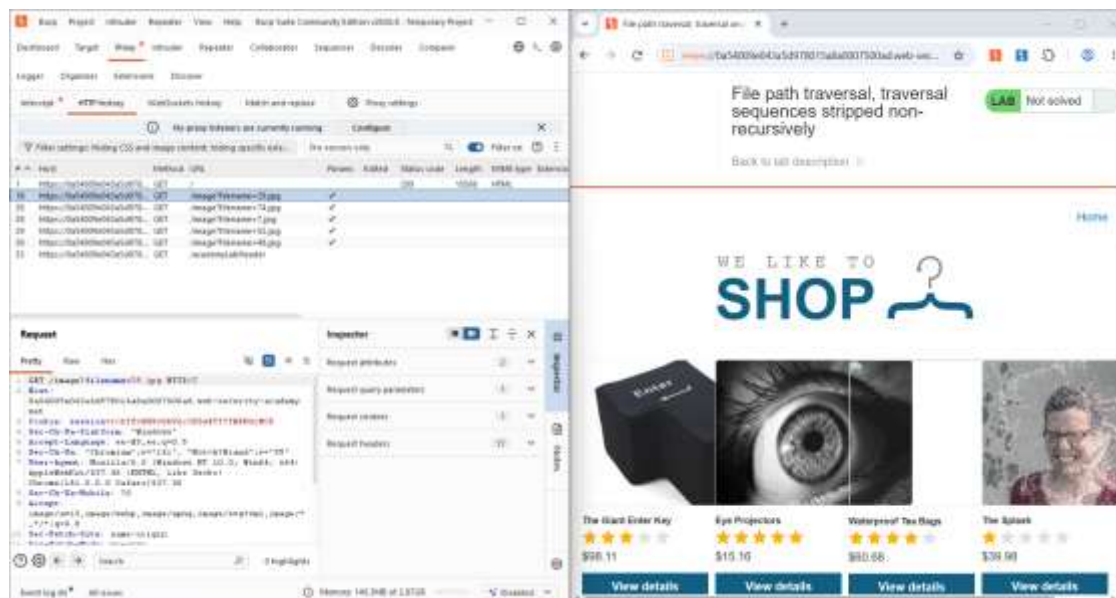


Figura 3. Historial de peticiones GET /image?filename=N.jpg en Burp Proxy.

5. Fase 2: Envío a Repeater y construcción del payload

Se seleccionó una de las peticiones de imagen y se envió a Repeater mediante “Send to Repeater” para poder modificar el parámetro filename de forma controlada.

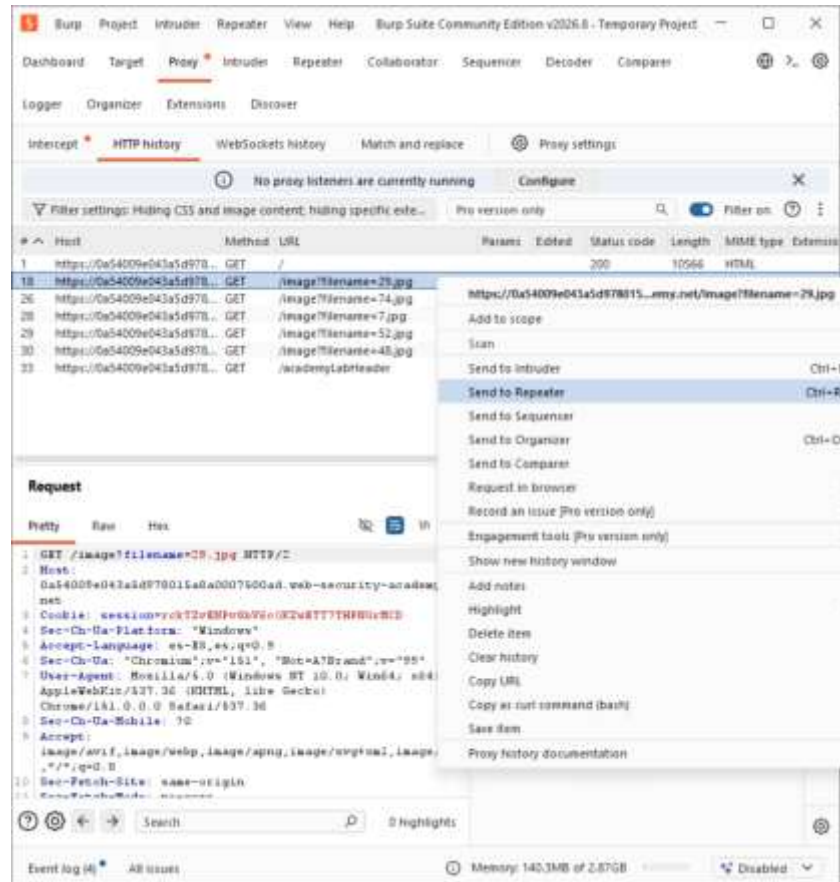


Figura 4. Envío de la petición seleccionada (filename=29.jpg) a Burp Repeater.

Dentro de Repeater se confirmó el valor original del parámetro, seleccionado en el Inspector, antes de proceder a modificarlo.

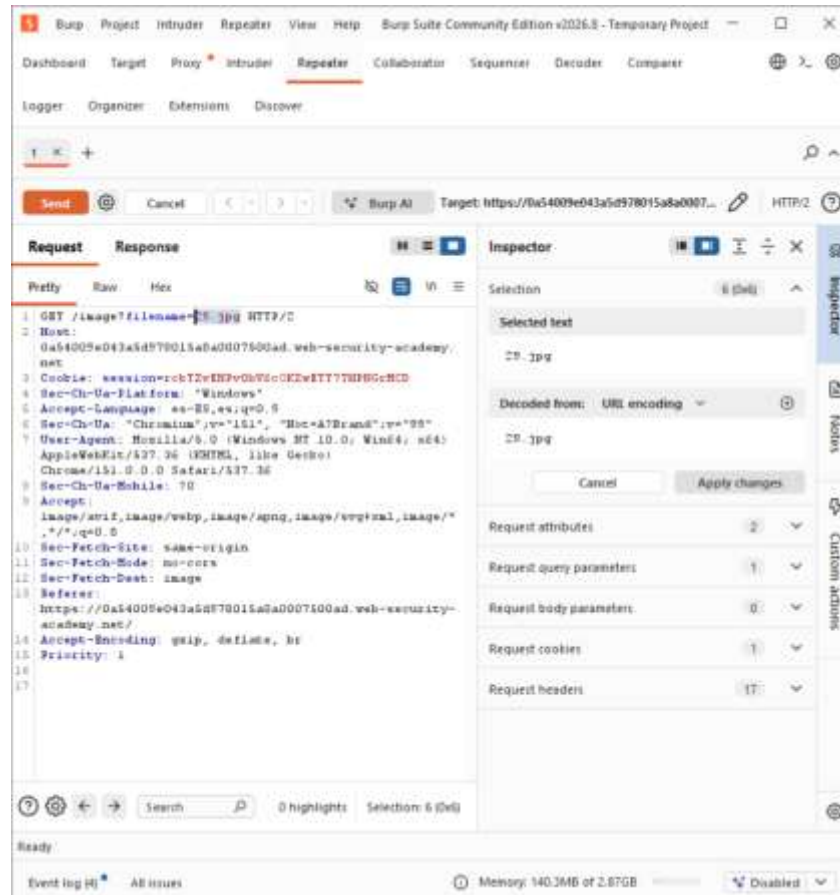


Figura 5. Petición original GET /image?filename=29.jpg en Repeater, con el valor resaltado en el Inspector.

Se construyó el siguiente payload, diseñado específicamente para evadir un filtro de eliminación no recursiva de “./”:

```
GET /image?filename=.../.../.../etc/passwd HTTP/2
```

6. Fase 3: Explotación y confirmación

Al enviar la petición con el payload construido, el servidor respondió con un código HTTP 200 OK y, en el cuerpo de la respuesta, con el contenido íntegro del archivo `/etc/passwd`.

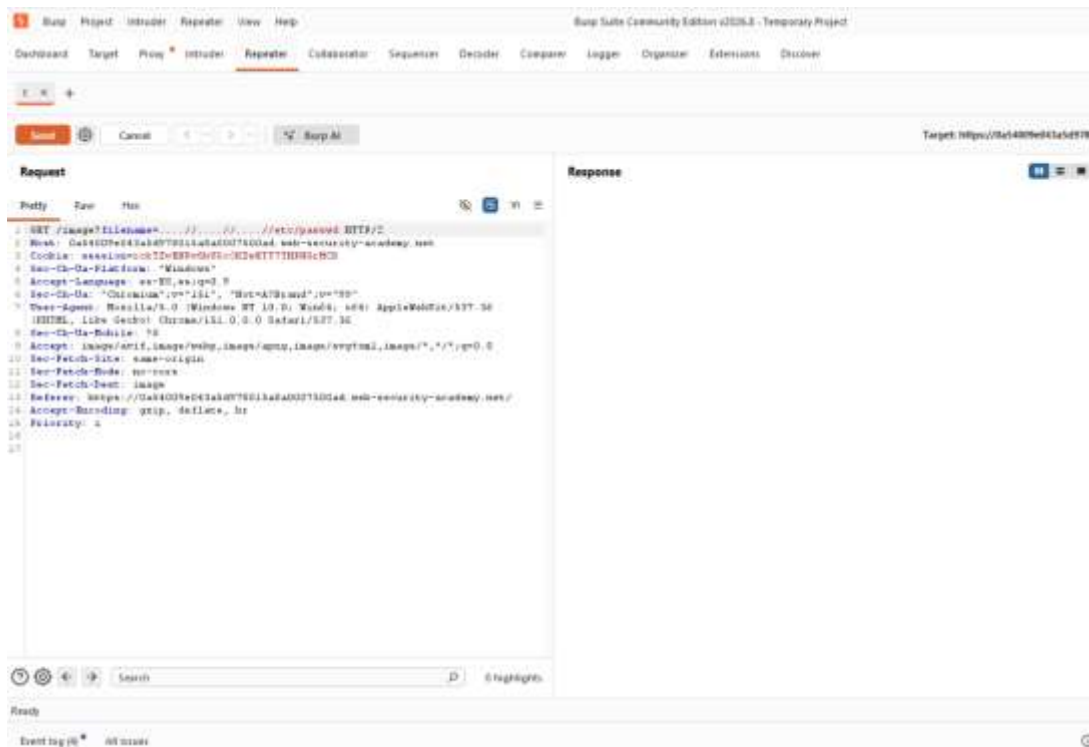


Figura 6. Petición modificada en Repeater con el payload//....//....//etc/passwd.

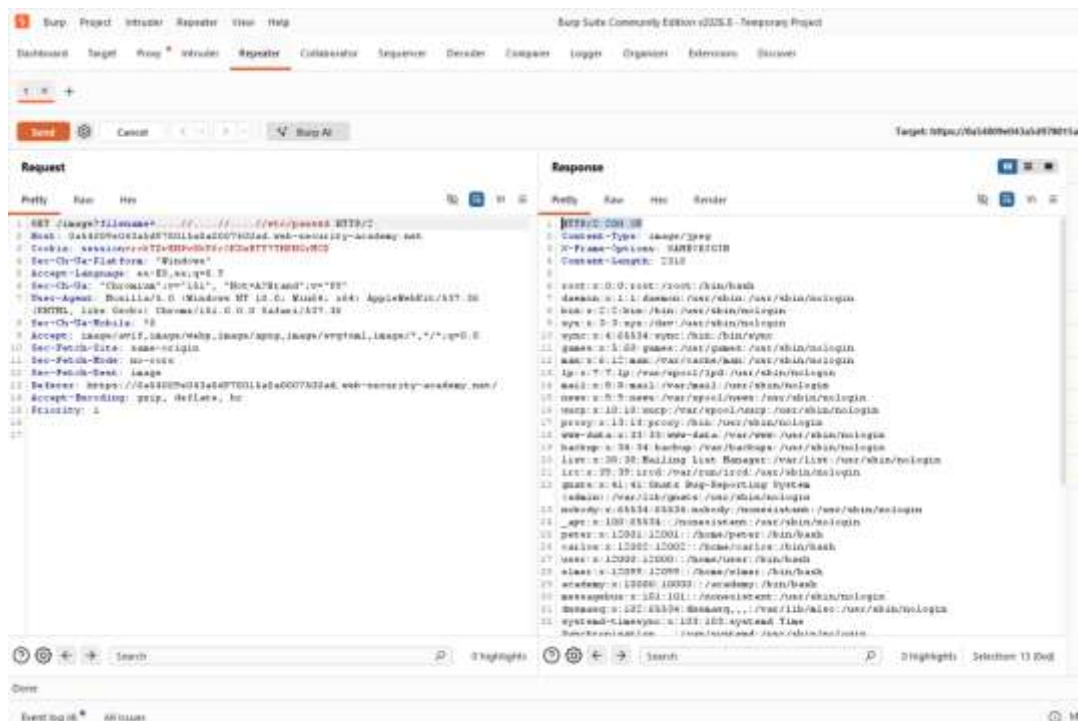


Figura 7. Respuesta del servidor mostrando el contenido completo de /etc/passwd.

6.1 Explicación del payload

El payload `....//....//....//etc/passwd` está diseñado para engañar a un filtro que elimina la subcadena `"../"` una única vez por cada aparición, sin repetir el proceso sobre el resultado. El razonamiento es el siguiente:

- Cada segmento `"....//"` contiene, intercalada, la secuencia `"../"`: si se toman los caracteres 2 a 4 de `"....//"` (`"../"`)... en realidad, el filtro típico busca literalmente la subcadena `"../"` dentro de `"....//"` y la elimina, dejando como remanente exactamente `"../"`.
- Es decir, al escribir cuatro puntos seguidos de dos barras (`"....//"`), el filtro detecta y elimina los tres caracteres centrales que coinciden con `"../"`, pero los caracteres restantes (un punto antes y una barra después) se recombinan automáticamente en una nueva secuencia `"../"` válida.
- Al repetirse este patrón varias veces (`....//....//....//`), el resultado tras el filtrado de una sola pasada es exactamente `../..../`, suficiente para escapar del directorio de imágenes y llegar a la raíz del sistema de archivos.

Finalmente, `etc/passwd` completa la ruta hacia el archivo objetivo, de forma idéntica a los payloads clásicos de path traversal.

Concepto teórico: bypass de filtros de una sola pasada. Esta técnica es un caso particular de una categoría más amplia de evasiones de sanitización: cuando un filtro de reemplazo o eliminación de un patrón no se aplica de forma recursiva es posible “anidar” el patrón dentro de sí mismo para que, tras una única eliminación, el remanente reconstruya el patrón original. La misma lógica aplica a otros contextos de sanitización de texto (por ejemplo, filtros de XSS o de inyección SQL) y subraya la importancia de aplicar la sanitización de forma iterativa hasta alcanzar un punto fijo, o de rechazar directamente la entrada en lugar de intentar “limpiarla”.

6.2 Resolución del laboratorio

Tras confirmar que la respuesta contenía el archivo `/etc/passwd`, PortSwigger marcó el laboratorio como resuelto.

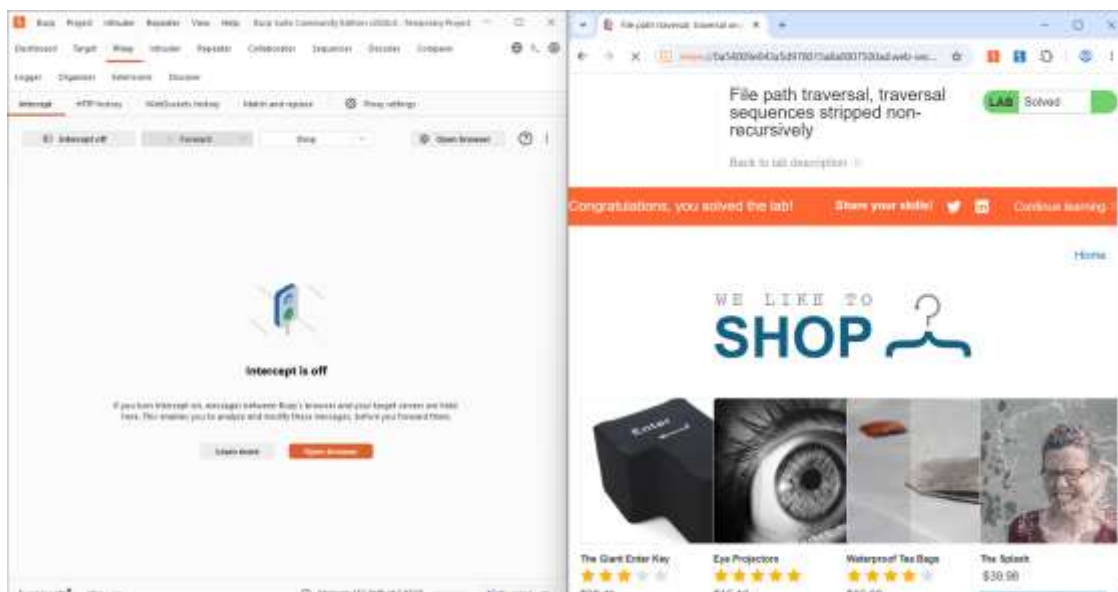


Figura 8. Confirmación de PortSwigger: “Congratulations, you solved the lab!”

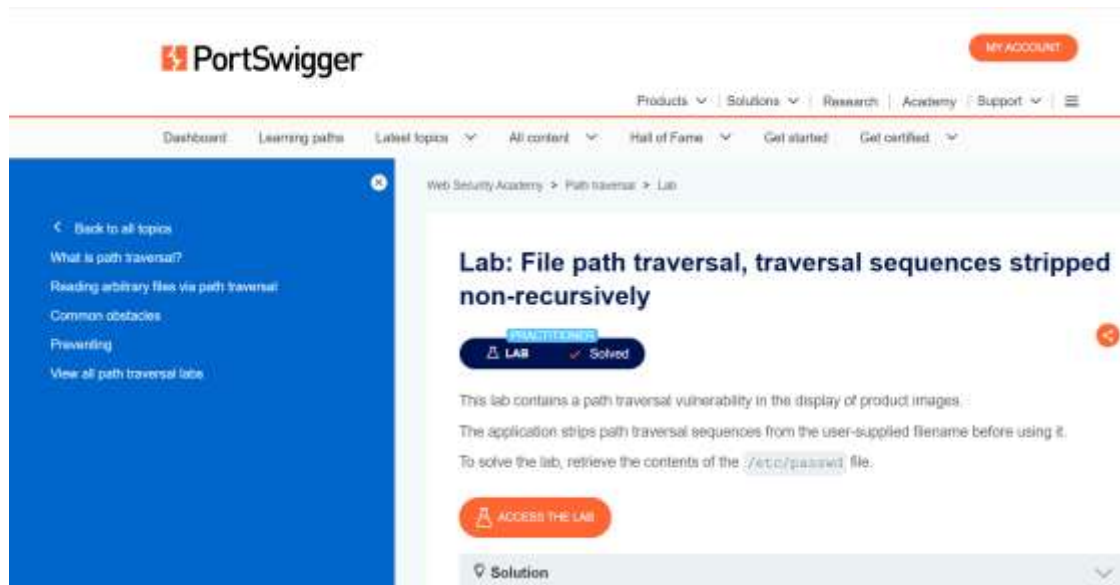


Figura 9. Estado final del laboratorio marcado como “Solved” en la página de descripción.

7. Mitigación recomendada

Para prevenir este tipo de bypass, se recomienda:

- Aplicar la eliminación de secuencias de traversal de forma recursiva o iterativa, repitiendo el proceso hasta que una nueva pasada no produzca ningún cambio en la cadena (alcanzar un punto fijo).
- Preferir el rechazo total de la entrada que contenga cualquier variante de “../” o “..\\”, en lugar de intentar sanearla o reescribirla.
- Resolver la ruta resultante a su forma canónica y verificar que se encuentre dentro del directorio base permitido antes de acceder al archivo.
- Sustituir el parámetro filename por un identificador indirecto validado contra una lista de archivos permitidos.
- Ejecutar el servicio con mínimo privilegio y aislar el proceso mediante chroot jail o contenedores.

8. Conclusiones

Este laboratorio evidenció que una defensa de sanitización aparentemente razonable puede volverse ineficaz si no se aplica de forma iterativa. El patrón//....//....//, sencillo en apariencia, es suficiente para reconstruir una secuencia de traversal válida tras una única pasada de filtrado, lo que subraya un principio general de seguridad: cuando sea posible, es preferible rechazar entradas sospechosas en lugar de intentar transformarlas o limpiarlas, ya que cualquier lógica de reescritura introduce una superficie adicional de posibles evasiones.